汇编语言

10 循环结构

张青
ieqzhang@zzu.edu.cn

实验四 数据处理类指令

1、首先判断下面每条指令执行后 AL 的数值和相关标志状态，写出结果。然后将其编辑成为一个完整的汇编语言源程序，可利用本书配套的输入输出子程序库，在每条指令之后，先调用其中的 DISPRF 显示标志位寄存器的值；然后再调用 DISPHD 显示 EAX 内容，并核对事先判断的结果。汇编连接、生成可执行文件。用 GDB 调试程序核对判断结果是否正确。

（1）mov esi,0b10011100 #ESI = _____H

and esi,0x80 #ESI = _____H

or esi,0x7f #ESI = _____H

xor esi,0xfe #ESI = _____H

（2）mov eax,0b1010 #EAX = _____B

shr eax,2 #EAX = _____B，CF = _____

shl eax,1 #EAX = _____B，CF = _____

and eax,3 #EAX = _____B，CF = _____

（3）mov eax,0b1011 #EAX = _____B

rol eax,2 #EAX = _____B，CF = _____

rcr eax,1 #EAX = _____B，CF = _____

or eax,3 #EAX = _____B，CF = _____

（4）

xor eax,eax #EAX = _____，CF = _____，

#OF = _____，ZF = _____，SF = _____

#PF = _____

实验四 数据处理类指令

2、编写计算 EAX×21 的移位指令实现程序。提示：$21 = 2^4 + 2^2 + 2^0$

3、写出 C 语言语句对应的汇编语言指令。编译生成对应的汇编语言程序，根据自动生成的汇编指令检查自己编写的汇编指令是否正确。将自己编写的汇编指令写成完整程序，汇编连接，记录运行结果。

```
long arith2(long x，long y，long z)
{
long t1 = x | y;
long t2 = t1 >> 3;
long t3 = ~t2;
long t4 = z-t3;
return t4;
}
```

教学内容

- 转移指令实现循环结构
- 循环指令实现循环结构

教材章节：5.2
慕课：8

- ▶循环指令
- ▶计数控制循环
- ▶条件控制循环

学习目标

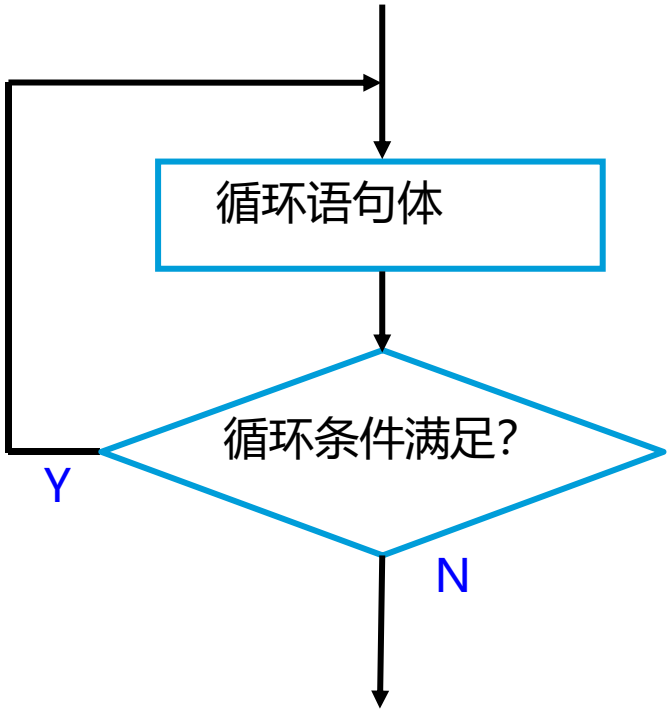
- 能说出转移指令的功能和格式
- 会正确判断转移发生的条件
- 记忆循环指令loop的功能
- 会阅读分析循环结构程序的功能、运行结果
- 会正确使用转移指令实现循环结构
- 会正确使用循环指令实现循环结构
- 会用gdb查看寄存器和内存
- 能说出程序开发过程中出现问题的原因并解决
- 养成良好的编程习惯，养成细心、严谨的工作态度
- 养成团队合作，互相帮助的精神

循环结构

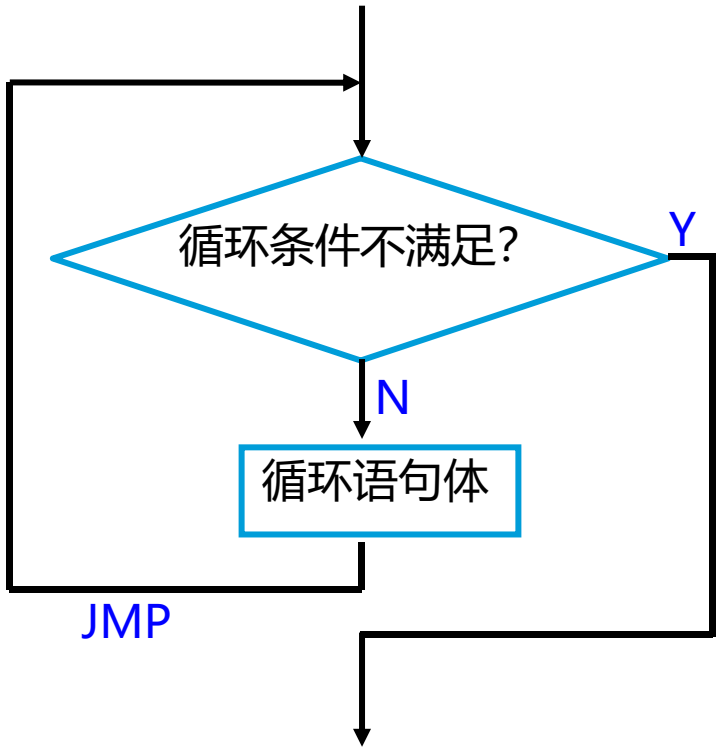
循环程序结构

- 三个部分组成：
  - 循环初始——为开始循环准备必要的条件，如循环次数、循环体需要的初始值等；
  - 循环体——重复执行的程序代码，其中包括对循环条件的修改等；
  - 循环控制——判断循环条件是否成立，决定是否继续循环
- “先判断、后循环”的循环程序结构
  - 对应高级语言的WHILE语句
- “先循环、后判断”的循环程序结构
  - 对应高级语言的DO语句
- 计数循环
  - 对应高级语言的for语句

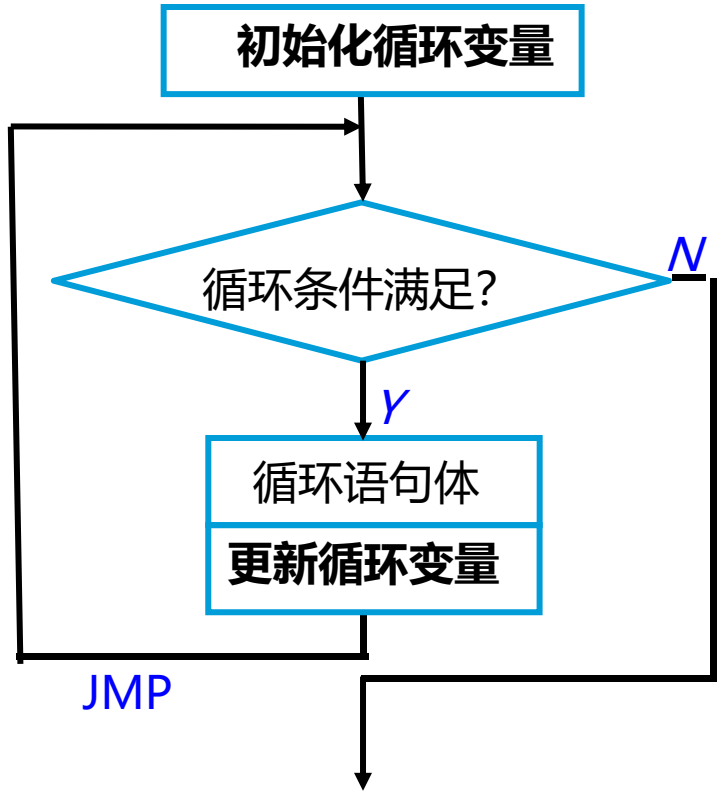
循环程序结构的流程图



(a) “先循环、后判断”结构



(b) “先判断、后循环”结构



(c)for循环结构

用转移指令实现循环结构

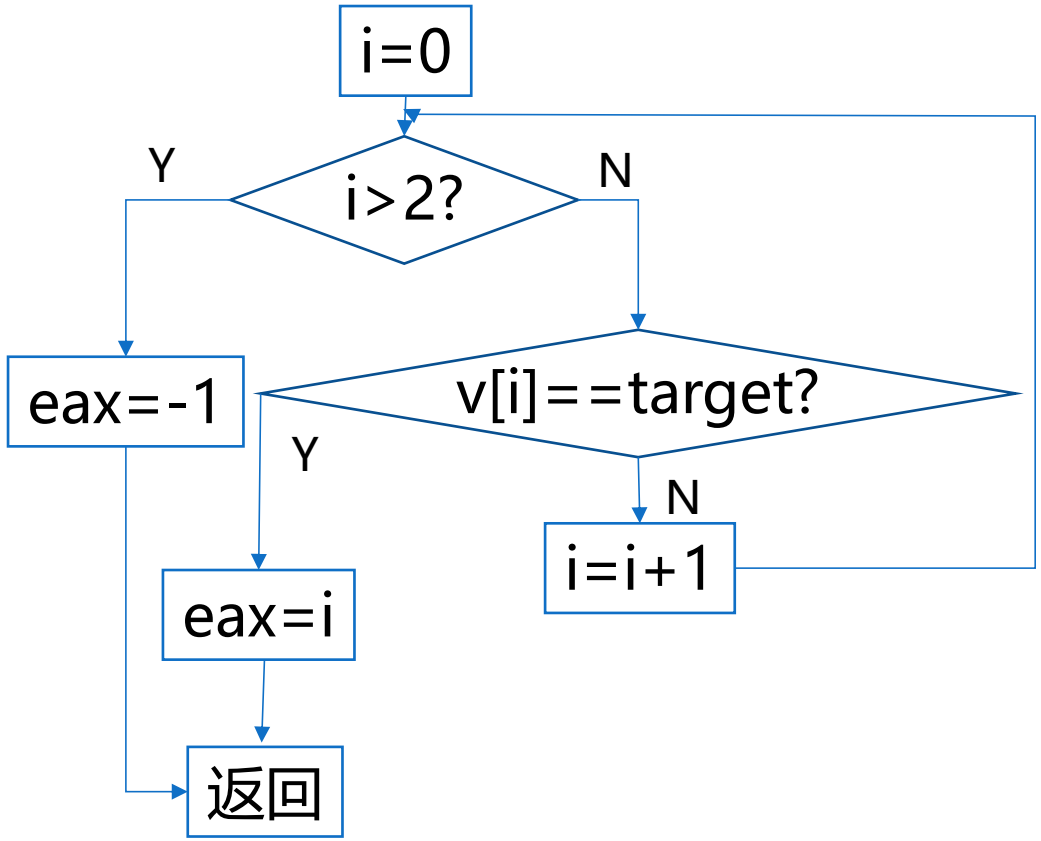
```
int find(int v[3], int target) {
    int i;
    for (i = 0; i < 3; i++) {
        if (v[i] == target) {
            goto found;
        }
    }
   return -1;

found:
    return i;
}
```

```
find: #rcx=数组首地址v，edx=target
        mov   eax, 0          #eax=i
        jmp   .L6
.L10:   add   eax, 1
.L6:    cmp   eax, 2
        jg    .L9
        movsx  r8, eax
        cmp   DWORD PTR [rcx+r8*4], edx
        jne   .L10
        jmp   .L5             #找到，eax=下标
.L9:    mov   eax, -1 #循环次数结束，没找到
.L5:    ret
```

find函数的汇编语言实现

int find(int v[3], int target)；

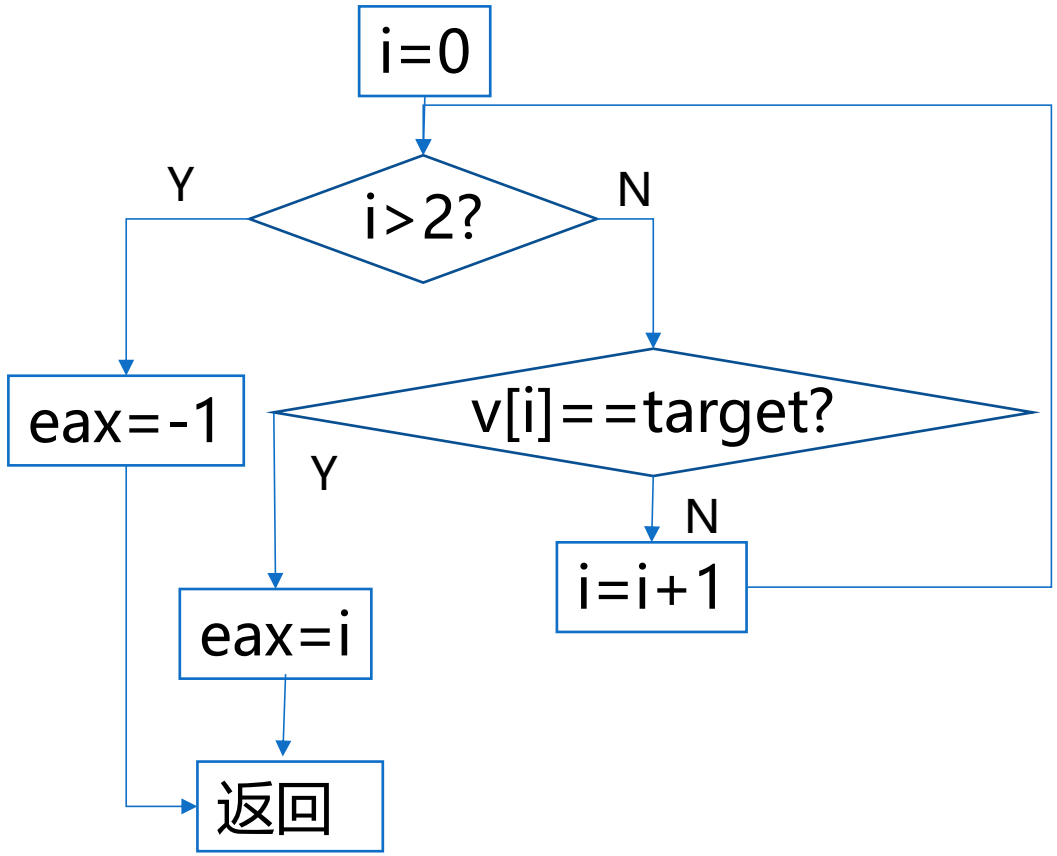
```
find: #rcx=数组首地址，edx=n
        mov    eax, 0  #初始化i
        jmp    .L6
.L10:   add    eax, 1
.L6:    cmp    eax, 2
        jg     .L9
        movsx  r8, eax
        cmp    DWORD PTR [rcx+r8*4], edx
        jne    .L10
        jmp    .L5      #找到，eax=下标
.L9:    mov    eax, -1 #循环次数结束，没找到
.L5:    ret
```



find函数汇编语言实现的简化

int find(int v[3], int target)；

```
find: #rcx=数组首地址，edx=n
        mov    eax, 0 #初始化i
.L6:    cmp    eax, 2
        jg     .L9
        movsx  r8, eax
        cmp    DWORD PTR [rcx+r8*4], edx
        jne    .L10
        jmp    .L5
.L10:   add    eax, 1
        jmp .L6
.L9:    mov    eax, -1
.L5:    ret
```



改变指令顺序可能会改变程序执行速度，这与指令流水线机制有关

〔例5-7〕n的阶乘-while循环

```
int fac(int n)
{
    int result = 1;
    while (n > 1) {
        result = result * n;
        n--;
    }
    return result;
}
```

```
fac: #-Og，  ecx=n
mov    eax, 1              # eax = result =1;
jmp    .L2
.L3:   imul   eax, ecx     # 循环体，ecx = n
sub    ecx, 1              # n-1
.L2:   cmp    ecx, 1       # 判断循环条件 n >1
jg     .L3                 # 满足循环条件
ret
```

〔例5-7〕人工汇编

```
fac: #-Og，ecx=n
mov    eax, 1              # eax = result =1;
jmp    .L2
.L3:   imul   eax, ecx     # 循环体，ecx = n
sub    ecx, 1              # n-1
.L2:   cmp ecx, 1          # 判断循环条件 n >1
jg     .L3                 # 满足循环条件
ret
```

```
fac: #人工修改
       mov    eax, 1       # eax = result =1;
.L2:   cmp    ecx, 1       # 判断循环条件 n >1
       jng    .L4          # 不满足循环条件
.L3:   imul   eax, ecx     # 循环体，ecx = n
       sub    ecx, 1       # n-1
       jmp .L2
 .L4:  ret
```

〔例5-8〕n的阶乘-do循环

```
int fac(int n)
{
    int result = 1;
    do {
        result = result * n;
        n--;
    }while(n>1);
    return result;
}
```

```
fac: #ecx=n
    mov   eax, 1
.L2:
    imul   eax, ecx
    sub    ecx, 1
    cmp   ecx, 1
    jg  .L2
    ret
```

〔例5-9〕n的阶乘-for循环

```
int fac(int n)
{
    int result = 1;
    for(int i=1;i<=n;i++)
        result = result * i;
    return result;
}
```

```
fac: #ecx:  n
     mov     eax, 1      # eax =i =1
     mov     edx, 1      # edx =result =1
     jmp     .L2
.L3:  imul    edx, eax    #循环体
     add     eax, 1
.L2: cmp     eax, ecx    # 判断循环条件i<=n
     jle     .L3         # 满足条件，执行循环体
     mov     eax, edx
     ret
```

〔例5-10〕字符个数统计程序-指针法

```
int mystrlen(char* str){
int n=0;
while(*str) n++;
return n;
}
```

```
mystrlen:# rcx=str
        mov   eax, 0                  # eax = n =0
        jmp   .L2
.L3:    add   eax, 1                  # 循环体
        mov   rcx, rdx                #rcx=rcx+1
.L2:    lea   rdx, 1[rcx]
        cmp   BYTE PTR [rcx], 0       # 循环条件 *str=0? rcx=str
        jne   .L3                     #不等于0，执行循环体
        ret
```

〔例5-10〕字符个数统计程序-下标法

```
int mystrlen1(char* str){
int n=0;
while(str[n]) n++;
return n;
}
```

```
mystrlen1:# rcx=str
        mov    eax, 0                    # eax = n =0
        jmp    .L5
.L6:    add    eax, 1
.L5:    movsx   rdx, eax
        cmp    BYTE PTR [rcx+rdx], 0 # 循环条件 *str=0?
        jne    .L6
        ret
```

〔例5-11〕斐波那契数列程序

生成斐波那契数列（不超过32位表示范围）

F(1)=1

F(2)=1

F(N)=F(N-1)+F(N-2)　　N≥3

- 用递归实现
- 用递推实现

〔例5-11〕指针法

```
void fib1(int fibs[],int n){
int *endfibs=fibs+n;
*fibs++=1;
*fibs++=1;
for(; fibs<endfibs; fibs++)
   *fibs = *(fibs-1) +*(fibs -2);
}
```

```
fib1:# edx =n，  rcx = fibs
       movsx  rdx, edx
       lea       r8, [rcx+rdx*4]
       #r8=fibs+n,每个数组元素为四字节
       mov     DWORD PTR [rcx], 1
       # DWORD PTR指定数组元素为4字节
       lea      rax, 8[rcx]      # rax = fibs+2
       mov    DWORD PTR 4[rcx], 1
       jmp            .L2
.L3: #循环体，  edx =fibs-2
       mov    edx, DWORD PTR -8[rax]
       # edx = *(fibs-1) +*(fibs -2)
       add      edx, DWORD PTR -4[rax]
       mov     DWORD PTR [rax], edx
       add      rax, 4   #下一个元素的地址
.L2: cmp      rax, r8  #循环条件
        jb       .L3  #满足循环条件，执行循环体
        ret
```

〔例5-11〕下标法

```
void fib2(int fibs[],int n){
fibs[0]=1;
fibs[1]=1;
for(int i=2;i<n; i++)
   fibs[i] = fibs[i-1] + fibs[i-2];
}
```

```
fib2: #eax = i,   edx =n,  rcx = fibs
          #初始化
   mov  DWORD PTR [rcx], 1
   mov  DWORD PTR 4[rcx], 1
   mov  eax, 2
   jmp   .L5        #转到循环进行条件判断
   .L6:   movsx  r8, eax  #循环体,,   r8=eax=i
   # r9d=fibs[i-2]
   mov  r9d, DWORD PTR -8[rcx+r8*4]
   # r9d=fibs[i]=fibs[i-1]+fibs[i-2]
   add   r9d, DWORD PTR -4[rcx+r8*4]
   mov  DWORD PTR [rcx+r8*4], r9d
   # r9d送回数组
   add   eax, 1
   .L5:   cmp     eax, edx  #循环条件,
   jl       .L6       #满足循环条件，执行循环体
   ret
```

- **循环指令**

Jecxz

loop

1 循环指令

LOOP label

#RCX/ECX/CX←RCX/ECX/CX - 1；若RCX/ECX/CX≠0，循环到LABEL

#否则，顺序执行

```
DEC RCX/ECX/CX
JNZ label
```

LOOPZ/LOOPE label

RCX/ECX/CX←RCX/ECX/CX - 1 ；

RCX/ECX/CX≠0且ZF=1，循环到label

#否则，顺序执行

```
JNZ  not_label
DEC RCX/ECX/CX
JNZ label
```

LOOPNZ/LOOPNE label

RCX/ECX/CX←RCX/ECX/CX - 1 ；

#若RCX/ECX/CX≠0且ZF=0，循环到label

#否则，顺序执行

```
JZ not_label
DEC RCX/ECX/CX
JNZ label
```

- 目标地址采用相对短转移
- 实模式：CX，32位保护模式：ECX，64位模式：RCX

循环指令测试

ECX=100, 语句 “delay: loop delay”会执行多少次？

ECX=1, 语句 “delay: loop delay”会执行多少次？

ECX=0, 语句 “delay: loop delay”会执行多少次？

ECX=-1, 语句 “delay: loop delay”会执行多少次？

语句 “delay: loop delay”会执行多少次?

- ECX=100　　100次
- ECX=1　　1次
- ECX=0　　2^{32}次
- ECX=-1　　2^{32}-1次

LOOP指令的循环次数由ECX决定

JECXZ指令

- 为避免计数初值为0可能导致的程序错误
- 设计JECXZ指令

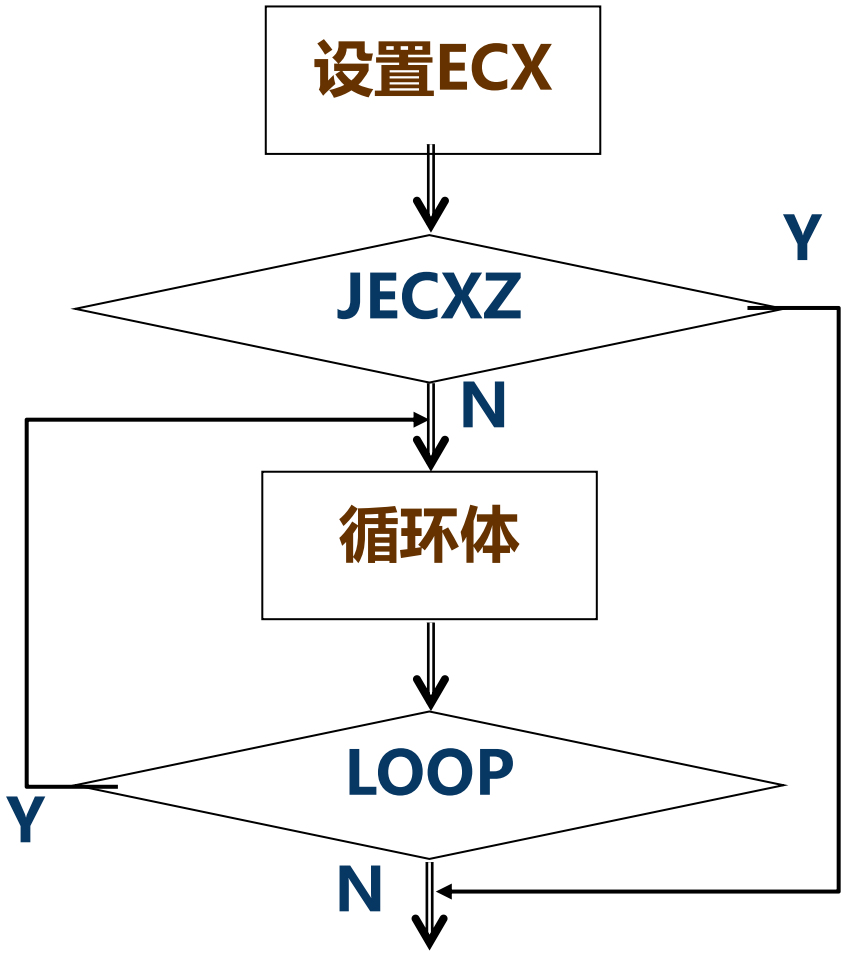
JECXZ label

;ECX = 0，转移到label

;否则，顺序执行

```
CMP ECX,0
JZ label
```

LOOP和JECXZ指令的典型应用



C语言编译器通常不会生成loop指令构成的循环体

〔例5-12〕n的阶乘-LOOP版

```
#eg0512.S
.intel_syntax noprefix      #采用Intel语法
      .data                 #定义数据段
      N: .long 10
      .text                 #定义代码段
     .globl main
     main:                  # 程序执行起始位置
       mov eax, 1
       mov ecx, N[rip]
again: imul eax, ecx
        loop again
        call dispsid        #调用子程序输出eax的值
        mov eax,0
        ret                 # 程序正常执行结束
```

N=100时，该程序运行结果错误

```
gcc –Wa,-alm –c – o eg0512.o ./progs/eg0512.S>./progs/eg0512.l
或：as –c – o eg0512.o ./progs/eg0512.S>./progs/eg0512.l
gcc – o eg0512 eg0512.o ./lib/winio64.a(io_linux64.a)
```

〔例5-12〕修改-增加溢出判断

```
.data                          #定义数据段
        N: .long 10
        error: .ascii "overflower\n"
             .byte 0
        .text                  #定义代码段
       .globl main
main:                          # 程序执行起始位置
        mov eax, 1
        mov ecx, N[rip]
again:  imul eax, ecx
        cmp eax,0              # 判断是否溢出
        js L1                  # 如果溢出则退出循环
        loop again
 call dispsid
 L2:   mov eax,0
        ret                    # 程序正常执行结束
 L1: lea rax, error[rip]  # 输出错误信息
 call dispmsg
 jmp L2
```

N=0，会出现死循环。

〔例5-12〕修改-防止死循环

```
main:                          # 程序执行起始位置
        mov eax, 1
        mov ecx, N[rip]
       cmp ecx, 0              #判断N=0
       jz L1                   #相当于jecxz L1
again:imul eax, ecx
        cmp eax,0              # 判断是否溢出
        js L1                  # 如果溢出则退出循环
        loop again
 L2:    mov eax,0
        ret                    # 程序正常执行结束
 L1: lea rax, error[rip]       # 输出错误信息
      call dispmsg
       jmp L2
```

〔例5-13〕求最大值程序

假设数组ARRAY由32位有符号整数组成，元素个数已知，没有排序。现要求编程获得其中的最大值。

```
.data                                   #定义数据段
 #假设一个数组
array:  .long -3,0,20,900,587,-632,777,234,-34,-56
.equ count, (. - array)/4      #数组的元素个数
max:     .space  4                      #存放最大值
```

〔例5-13〕求最大值程序

```
        .text                   #定义代码段
      .globl main
main:                           # 程序执行起始位置
        mov ecx, count-1        #元素个数减1是循环次数
        lea rsi, array[rip]
        mov eax,[rsi]           #取出第一个元素给EAX，用于暂存最大值
again: add rsi,4
        cmp eax,[rsi]           #与下一个数据比较
        jge next                #已经是较大值，继续下一个循环比较
        mov eax,[rsi]           #EAX取得更大的数据
next:   loop again              #计数循环
        mov max[rip],eax        #保存最大值
       call dispsid             #显示结果
```

〔例5-14〕简单加密解密程序

- 用户的明文逐个数据与密钥Y进行异或，实现加密。
- 加密后的密文再次与Y进行异或就实现解密。

```
            .data                          #定义数据段
             key: .byte 234                #假设的一个密钥
msg1:  .ascii "This is a screte."          #源字符串
            .equ count , .-msg1
            .byte 0
msg2:  .ascii "Encrypted message:\0 "
msg3:   .ascii "\nOriginal messge:\0"
```

〔例5-14〕简单加密解密程序-加密

```
        .text                   #定义代码段
        ......
        #ECX = 实际输入的字符个数，作为循环的次数
        mov ecx, count
        xor rbx,rbx             #RBX清零，指向源字符串中的每个字符
        mov al,key[rip]         #AL＝密钥
        lea rsi,msg1[rip]
encrypt:
        xor [rsi+rbx],al        #异或加密
        inc rbx                 #指向下一个字符
        loop encrypt            #相当于
        lea rax, msg2[rip]
        call dispmsg
        lea rax,msg1[rip]       #显示加密后的密文
        call dispmsg
```

```
dec ecx
jnz encrypt
```

〔例5-14〕简单加密解密程序-解密

```
        #ECX＝实际输入的字符个数，作为循环的次数
        mov ecx, count
        xor rbx,rbx              #RBX清零，指向加密后字符串中的每个字符
        mov al,key[rip]          #AL＝密钥
decrypt:xor [rsi+rbx],al         #异或解密
        inc rbx
        loop decrypt             #相当于
        lea rax, msg3[rip]
        call dispmsg
        lea rax, msg1[rip]       #显示解密后的明文
        call dispmsg
```

```
dec ecx
jnz decrypt
```

〔例5-14〕优化

采用反向访问（从最后一个元素到第一个元素）的顺序，可以直接利用rcx作为字符位置索引，可优化汇编指令序列

```
main:                                   # 程序执行起始位置
        #ECX = 实际输入的字符个数，作为循环的次数
        mov ecx, count
        #xor rbx,rbx ——→ 删去
        mov al, key[rip]        #AL = 密钥
        lea rsi,msg1[rip]
encrypt: xor -1[rsi+rcx],al   #异或加密
        #inc rbx      ——→ 删去
        loop encrypt            #处理下一个字符
```